

Chapter 7. Documentation and Understanding

A production-ready microservice is documented and understood. Documentation and organizational understanding increase developer velocity while mitigating two of the most significant trade-offs that come with the adoption of microservice architecture: organizational sprawl and technical debt. This chapter explores the essential elements of documenting and understanding a microservice, including how to build comprehensive and useful documentation, how to increase microservice understanding at every level of the microservice ecosystem, and how to implement production-readiness throughout an engineering organization.

Principles of Microservice Documentation and Understanding

I'm going to open this final chapter on the last principle of microservice standardization with a famous story from Russian literature. While it may seem rather unorthodox to quote Dostoyevsky in a book on software architecture, the character Grushenka in *The Brothers Karamazov* captures so perfectly what I believe to be the key of microservice documentation and understanding: “Just know one thing, Rakitka, I may be wicked, but still I gave an onion.”

My favorite part of Dostoyevsky's brilliant novel is a tale told by the character Grushenka about an old woman and an onion. The tale goes something like this: there was once an old, bitter woman who was very selfish and heartless. One day, she happened upon a beggar, and for some reason, felt a great deal of pity. She wanted to give something to the beggar, but all she had was an onion, so she gave her onion to the beggar. The old woman eventually died, and thanks to her bitterness and coldness of heart, ended up in hell. After she had suffered for quite some time, an angel came to save her, for God had remembered her one selfless deed in life, and wanted to extend the same kindness in return. The angel reached out to her with an onion in his hand. The old woman grabbed the

onion, but to her dismay, the other sinners around her reached for the onion too. Her cold, bitter nature kicked in, and she tried to fight them off, not wanting any of them to have any piece of the onion, and sadly, as she tried to claw the onion away from them, the onion split into many layers and she and the other sinners fell back into hell.

It's not the most heartwarming tale, but there's a moral to Grushenka's story that I have found remarkably applicable to the practice of microservice documentation: always give an onion.

The importance of thorough, updated documentation for every microservice cannot be emphasized enough. Ask developers working in a microservice ecosystem what their main concerns are, and they'll rattle off a list of features still to be implemented, bugs to be fixed, dependencies that are causing trouble, and things that they don't understand about their own service and the dependencies they rely on. When asked to go into greater detail about the latter two things, they tend to give similar answers: they don't understand how it works, it's a black box, and the documentation is completely useless.

Poor documentation of dependencies and internal tools slows developers down and affects their ability to make their own services production-ready. It prevents them from using dependencies and internal tools correctly and wastes countless engineering hours, because sometimes the only way to figure out what a service or tool does (without proper documentation) is to reverse-engineer it until you understand how it works.

Poor documentation of a service also hurts the productivity of the developers who are contributing to it. For example, the lack of runbooks for an on-call shift means whoever is on call will need to figure out each problem from square one every single time. Without an onboarding guide, each new developer working on the service will need to start from scratch to understand how the service works. Single points of failure and problems with the service will go unnoticed until they cause an outage. New features added to the service will often miss the big picture of how the service actually works.

The goal of good, production-ready documentation is to create and maintain a centralized repository of knowledge about the service. Sharing that knowledge has two components: the bare facts about the service, and or-

ganizational understanding of what the service does and where it fits into the organization as a whole. The problem of poor documentation can then be divided into two subproblems: lack of documentation (the facts) and lack of understanding. Solving these two subproblems requires standardizing documentation for every microservice and putting organizational structures into place for sharing microservice understanding.

Grushenka's tale is the golden rule of microservice documentation: always give an onion. Give an onion for your sake, for the sake of fellow developers working on your service, and for the sake of the developers whose services depend on yours.

A PRODUCTION-READY SERVICE IS DOCUMENTED AND UNDERSTOOD

- It has comprehensive documentation.
 - Its documentation is updated regularly.
 - Its documentation contains a description of the microservice; an architecture diagram; contact and on-call information; links to important information; an onboarding and development guide; information about the service's request flow(s), endpoints, and dependencies; an on-call runbook; and answers to frequently asked questions.
 - It is well understood at the developer, team, and organizational levels.
 - It is held to a set of production-readiness standards and meets the associated requirements.
 - Its architecture is reviewed and audited frequently.
-

Microservice Documentation

The documentation for all microservices in an engineering organization should be stored in a centralized, shared, and easily accessible place. Any developer on any team should be able to find the documentation for every microservice without any difficulty. An internal website containing the documentation for all microservices and internal tools tends to be the best medium for this.

Many developers limit the documentation of their microservices to a README file in their repository or to comments scattered throughout the code. While having a README is essential, and all microservice code should contain appropriate comments, this is *not* production-ready documentation and requires that developers check out and search through the code. Proper documentation is stored in a centralized place (like a website) where the documentation for all microservices in the engineering organization lives.

The documentation should be updated regularly. Any time a significant change is made to the service, the documentation should be updated. For example, if a new API endpoint is added, information about the endpoint must be added to the documentation as well. If a new alert is added, then step-by-step instructions on how to triage, mitigate, and resolve the alert should also be added to the service's on-call runbook. If a new dependency is added, then information about that dependency should be added to the documentation. Always give an onion.

The best way to accomplish this is to make the process of updating documentation part of the development workflow. If updating documentation is seen as a separate task aside from (and secondary to) development, then it will never get done and will become part of the technical debt of the service. To reduce technical debt, developers should be encouraged (or, if need be, required) to accompany every significant code change with an update to the documentation.

MAKE UPDATING DOCUMENTATION PART OF THE DEVELOPMENT CYCLE

If updating and improving documentation is viewed as secondary to writing code, it will often be pushed off and become part of the technical debt of the service. To avoid this, make documentation updates and improvements a required part of the development cycle of the service.

Documentation should be both comprehensive and useful. It should contain all of the relevant and important facts about the service. After reading through the documentation, a developer should know how to develop and contribute to the service; the architecture of the service; the contact and on-call information for the service; how the service works (request

flows, endpoints, dependencies, etc.); how to triage, mitigate, and fix incidents and outages as well as resolve alerts generated by the service; and answers to frequently asked questions about the service.

Most importantly, documentation should be written clearly and should be easy to understand. Jargon-heavy documentation is useless, documentation that is overly technical and doesn't explain things that may be unique to the service is also useless, as is documentation that doesn't go into any significant detail at all. The goal in writing good, clean, and clear documentation is to write it so that it can be understood by any developer, manager, product manager, or executive within the company.

Let's dive a little bit deeper into each of the elements of production-ready microservice documentation.

Description

Each microservice's documentation should begin with a *description* of the service. It should be short, sweet, and to the point. For example, if there is a microservice called *receipt-sender* whose purpose is to send a receipt after a customer completes an order, the description should read:

Description:

After a customer places an order, receipt-sender sends a receipt to the customer via email.

This is essential because it ensures that anyone who finds the documentation will know what role the microservice plays in the microservice ecosystem.

Architecture Diagram

The description of the service should be followed by an *architecture diagram*. This diagram should detail the architecture of the service, including its components, its endpoints, the request flow, its dependencies (both upstream and downstream), and information about any databases or caches. See an example architecture diagram in [Figure 7-1](#).

Architecture diagrams are essential for several reasons. It's nearly impossible to understand how and why a microservice works just by reading through the code, and so a well-designed architecture diagram is an easily understandable visual description and summary of the microservice. These diagrams also aid developers in adding new features by abstracting away the inner workings of the service so that developers can see where and how new features will (or will not) fit. Most importantly, they illuminate issues and problems with the service that would go unnoticed without a complete visual representation of its architecture: it's difficult to discover a service's points of failure by combing through lines of code, but they tend to stick out like sore thumbs in an accurate architecture diagram.

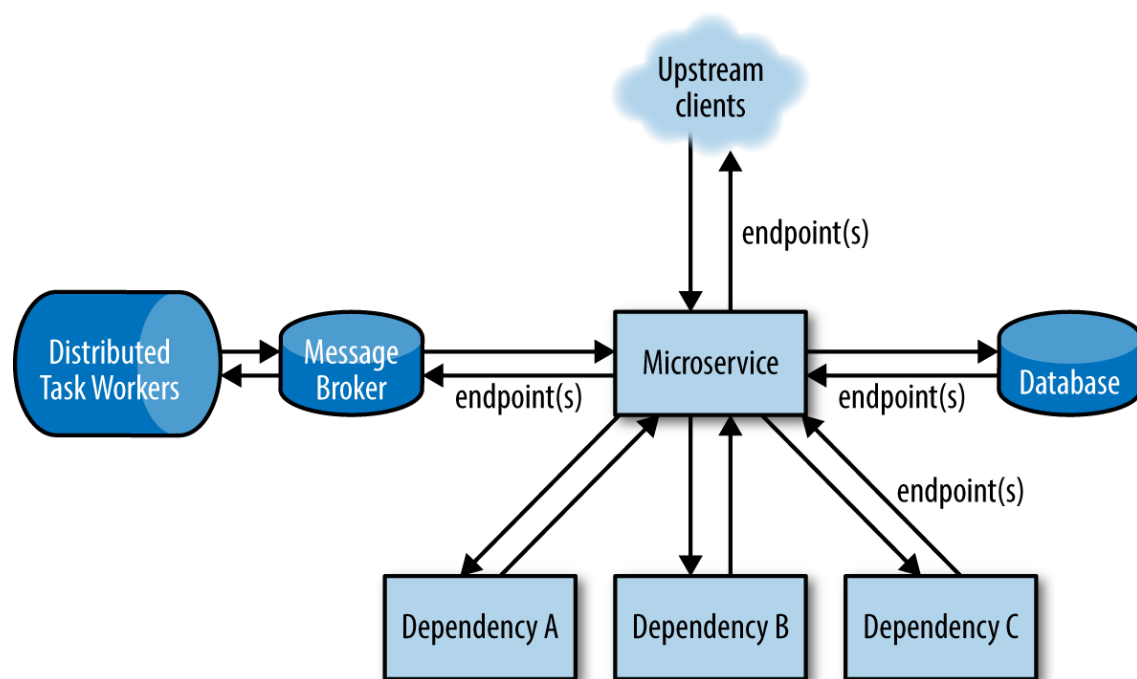


Figure 7-1. Example microservice architecture diagram

Contact and On-Call Information

Chances are, anyone looking at a service's documentation will either be someone on the service team, or someone on a different team who is experiencing trouble with the service or wants to know how the service works. For developers in the second group, having access to information about the team is both useful and necessary, and so several important facts should be included in a *contact and on-call information* section within the documentation.

This section should include the names, positions, and contact information of everyone on the team (including individual contributors, managers,

and program/product managers). This makes it easy for developers on other teams to quickly determine who they should contact if they experience a problem with the service or have a question about it. This information is useful, for example, when a developer is experiencing problems with one of their dependencies: knowing who to contact and what their role is on the team makes cross-team communication easy and efficient.

Adding information about the on-call rotation (and keeping it updated so that it reflects who is on call for the service at any given time) will ensure that people will know exactly who to contact for general problems or emergencies: the engineer who is on call for the service.

Links

Documentation needs to be a centralized resource for all the information about a microservice. In order for this to be true, the documentation needs to contain links to the repository (so that developers can easily check out the code), a link to the dashboard, a link to the original RFC for the microservice, and a link to the most recent architecture review slides. Any extra information about other microservices, technologies used by the microservice, etc., that may be useful to the developer should be included in a *links* section of the documentation.

Onboarding and Development Guide

The purpose of an *onboarding and development* section is to make it easy for a new developer to onboard to the team, begin contributing code, add features to the microservice, and introduce new changes into the deployment pipeline.

The first part of this section should be a step-by-step guide to setting up the service. It should walk a developer through checking out the code, setting up the environment, starting the service, and verifying that the service is working correctly (including all commands or scripts that need to be run in order to accomplish this).

The second part should guide the developer through the development cycle and deployment pipeline of the service (details of a production-ready development cycle and deployment pipeline can be found in [“The Devel-](#)

[opment Cycle](#)” and [“The Deployment Pipeline”](#)). This should include the technical details (e.g., commands that must be run, along with several examples) of each of the steps: how to check out the code, how to make a change to the code, how to write a unit test for the change (if necessary), how to run the required tests, how to commit their changes, how to send changes for code review, how to make sure that the service is built and released correctly, and then how to deploy (as well as how the deployment pipeline is set up for the service).

Request Flows, Endpoints, and Dependencies

The documentation should also contain critical information about *request flows, endpoints, and dependencies* of the microservice.

Request flow documentation can consist of a diagram of the request flows of the application. This can be the architecture diagram, if the request flow is detailed appropriately within the architecture diagram. Any diagram should be accompanied by a qualitative description of the types of requests that are made to the microservice and how they are handled.

This is also the place to document all API endpoints of the service. A bulleted list of the endpoints with their names and a qualitative description of each along with their responses is usually sufficient. It must be clear and understandable enough that another developer working on a different team could read the descriptions of your service’s API endpoints and treat your microservice as a black box, hitting the endpoints successfully and receiving the expected responses.

The third element of this section is information about the service’s dependencies. List the dependencies, the relevant endpoints of these dependencies, and any requests the service makes to them, along with information about their SLAs, any alternatives/caching/backups in place in case of failure, and links to their documentation and dashboards.

On-Call Runbooks

As covered in [Chapter 6, Monitoring](#), every single alert should be included in an on-call runbook and accompanied by step-by-step instructions describing how it should be triaged, mitigated, and resolved. The on-call

runbook should be kept in the centralized documentation of the service, in an *on-call runbook* section, along with both general and detailed guidance on troubleshooting and debugging new errors.

A good runbook will begin with any general on-call requirements and procedures, and then contain a complete list of the service's alerts. For each alert, the on-call runbook should include the alert name, a description of the alert, a description of the problem, and a step-by-step guide on how to triage the alert, mitigate it, and then resolve it. It will also describe any organizational implications of the alert: the severity of the problem, whether or not the alert signifies an outage, and information about how to communicate any incidents and outages to the team, and if necessary, to the rest of the engineering organization.

WRITE ON-CALL RUNBOOKS THAT SLEEPY DEVELOPERS CAN UNDERSTAND AT 2 A.M.

Developers on call for a service may (or, more realistically, will) be paged at any hour of the day, including late at night or very early in the morning. Write your on-call runbooks so that a half-asleep developer will be able to follow along without any difficulty.

Writing good, clear, easily understandable on-call runbooks is extremely important. They should be written so that any developer who is on call for the service or who is experiencing trouble with the service will be able to act quickly, diagnose the problem, mitigate the incident, and resolve, all in an extremely small amount of time in order to keep the downtime of the service very, very low.

Not every alert will be easily mitigated or resolved, and most outages (aside from those caused by code bugs introduced by a recent deployment) haven't been seen before. To equip developers to handle these problems wisely, add a general *troubleshooting and debugging* section to the on-call runbook in the documentation that is filled with tips on how to approach new problems in a strategic and methodical way.

FAQ

An often forgotten element of documentation is a section devoted to answering common questions about the service. Having a "Frequently

Asked Questions” section takes the burden of answering common questions off of whomever is on call and, consequently, the rest of the team.

There are two categories of questions that should be answered here. The first are questions that developers on other teams ask about the service. The way to approach answering these questions in an FAQ setting is simple: if someone asks you a question, and you think it might be asked again, add it to the FAQ. The second category of questions are those that come from team members, and the same approach can be taken here: if there’s a question about how or why or when to do something related to the service, add it to the FAQ.

SUMMARY: ELEMENTS OF PRODUCTION-READY MICROSERVICE DOCUMENTATION

Production-ready microservice documentation includes:

- A description of the microservice and its place in the overall microservice ecosystem and the business
- An architecture diagram detailing the architecture of the service and its clients and dependencies at a high level of abstraction
- Contact and on-call information about the microservice’s development team
- Links to the repository, dashboard(s), the RFC for the service, architecture reviews, and any other relevant or useful information
- An onboarding and development guide containing details about the development process, the deployment pipeline, and any other information that will be useful to developers who contribute code to the service
- Detailed information about the microservice’s request flows, SLA, production-readiness status, API endpoints, important clients, and dependencies
- An on-call runbook containing general incident and outage response procedures, step-by-step instructions on how to triage, mitigate, and resolve each alert, and a general troubleshooting and debugging section
- A “Frequently Asked Questions” (FAQ) section

Microservice Understanding

Centralized, updated, and thorough documentation is only one part of production-ready microservice documentation and understanding. Aside from writing and updating documentation, organizational processes should be put into place to ensure that microservices are well understood not only by the individual development teams but by the organization as a whole. In many ways, a well-understood microservice is one that meets every production-readiness requirement.

Microservice understanding is truly indispensable to the developer, the team, and the organization. While the notion of “understanding” a microservice may seem too vague to be useful at first glance, the concept of a production-ready microservice can be used to guide and define microservice understanding at every level. Armed with production-readiness standards and requirements, along with a realistic understanding of organizational complexity and the challenges that microservice architecture brings to the arena, developers can quantify their understanding of each microservice and (as I’ve urged the reader earlier in this chapter) can give an onion to the rest of the organization.

For the individual developer, this translates to being able to answer questions about her microservice. For example, when asked if her microservice is scalable, she will be able to look at a list of scalability requirements and confidently answer “Yes,” “No,” or something in between (e.g., “It meets requirements *x* and *z*, but *y* has not yet been implemented”). Likewise, when asked if her microservice is fault tolerant, she’ll be able to rattle off all failure scenarios and possible catastrophes, then explain in detail how she has prepared for these using various types of resiliency testing.

At the team level, *understanding* signifies that the team is aware of where their microservice stands with regard to production-readiness and what needs to be accomplished to bring their service to a production-ready state. This has to be a cultural element of each team in order for it to be successful: production-readiness standards and requirements need to drive the decisions made by the team and be seen not merely as boxes to check off on a checklist, but rather as principles that guide the team toward building the best possible microservice.

Understanding needs to be built into the fabric of the organization itself. This requires that production-readiness standards and requirements be-

come part of the organizational process. Before a service is even built, and a *request for comments* (RFC) is sent around for review, the service can be evaluated against the production-readiness standards and requirements. Developers, architects, and operations engineers can make sure that the service is built for stability, reliability, scalability, performance, fault tolerance, catastrophe-preparedness, proper monitoring, and appropriately documented and understood before it even begins running—ensuring that once the new service begins to host production traffic, it has been architected and optimized for availability and can be trusted with production traffic.

It's not enough to only review and architect for production-readiness at the beginning of a microservice's lifecycle. Existing services need to be reviewed and audited constantly so that the quality of each microservice is kept at a sufficiently high level, ensuring high availability and trust across various microservice teams and the entire microservice ecosystem. Automating these production-readiness audits of existing services and internally publicizing the results can help to establish awareness across the organization about the quality of the overall microservice ecosystem.

Architecture Reviews

One thing I've learned after driving these production-readiness standards and their requirements across over a thousand different microservices and their development teams is that the most immediately effective way to accomplish microservice understanding is to hold scheduled *architecture reviews* for each microservice. A good architecture review is a meeting where any and all developers and site reliability engineers (or other operations engineers) working on the service meet in a room, draw up the architecture of the service on a whiteboard, and thoroughly evaluate its architecture.

Within several minutes into this exercise, it tends to become very clear precisely what the scope of understanding is at the developer and team levels. Talking through the architecture, developers will quickly discover scalability and performance bottlenecks, previously undiscovered points of failure, possible outages and future incidents and failures and catastrophe scenarios, and new features that should be added. Poor architectural decisions that were made in the past will become obvious, and old tech-

nologies that should be replaced by newer and/or better ones will stand out. To ensure that evaluation and discussion is productive and objective, it's helpful to bring in developers from other teams (especially those in infrastructure, DevOps, or site reliability engineering) who have experience in large-scale distributed systems architecture (and the organization's specific microservice ecosystem) and will be able to point out problems that developers may not notice.

Each meeting should produce a new, updated architecture diagram for the service, along with a list of projects to tackle in the coming weeks and months. The new diagram should definitely be added to the documentation, and projects can be included in each service's *roadmap* (see [“Production-Readiness Roadmaps”](#)) and *objectives and key results* (OKRs).

Because microservice development moves rather quickly, microservices evolve at a rapid pace and the lower layers of the microservice ecosystem will be constantly changing. In order to keep the architecture and its understanding relevant and productive, these meetings should be held regularly. I've found that a good rule of thumb is to schedule them so that they align with OKR and project planning. If projects and OKRs are planned and scheduled quarterly, then quarterly architecture reviews should be held each quarter before the planning cycle begins.

Production-Readiness Audits

To make sure that a microservice meets production-readiness standards and requirements and is actually production-ready, the team can run a *production-readiness audit* on the service. Running an audit is quite simple: the team sits down with a checklist of the production-readiness requirements and checks off whether or not their service meets each requirement. This enables understanding of a service: each developer and team will know, by the end of the audit, exactly where their service stands and where things can be improved.

The structure of an audit should mirror the production-readiness standards and requirements that the engineering organization has adopted. The team should use the audits to quantify the stability, reliability, scalability, fault tolerance, catastrophe-preparedness, performance, monitoring, and documentation of the service. As I've described in earlier chap-

ters, each of these standards is accompanied by a set of requirements that can be used to bring each service up to those standards—developers can adjust these requirements of each production-readiness standard so that they meet the needs and goals of the organization. The exact requirements will depend on the details of the company’s microservice ecosystem, but the standards and their basic components are relevant across every ecosystem (see [Appendix A](#) for a summary checklist containing the production-readiness standards and their general requirements).

Production-Readiness Roadmaps

Once a microservice development team has completed a thorough production-readiness audit of their microservice and the team understands whether their service is production-ready, the next step is to plan how to bring the service to a production-ready state. Audits make this easy: at this point, the team has a checklist of which production-readiness requirements their service doesn’t meet, and all that is left to do is to satisfy each unfulfilled requirement.

This is where *production-readiness roadmaps* can be developed, and I’ve found them to be an extremely useful piece of the production-readiness and microservice understanding process. Each microservice is different, and the implementation details of each unsatisfied requirement will vary between services, so producing a detailed roadmap that documents all of the implementation details will guide the team toward making their microservice production-ready. Requirements that need to be met can be accompanied by the technical details, problems that have arisen (outages and incidents) that are related to the requirement, a link to some ticket in a task-management system, and the name(s) of the developer(s) who will be working on the project.

The roadmap and the list of unsatisfied production-readiness requirements it contains can become part of whatever planning and (if used at the company) OKRs are in store for the service. Satisfying production-readiness requirements works best when the process goes hand in hand both with feature development and with the adoption of new technologies. Making each service in the microservice ecosystem stable, reliable, scalable, performant, fault tolerant, catastrophe-prepared, monitored, documented, and understood is a straightforward, quantifiable way to

guarantee that each service is truly production-ready, ensuring the availability of the entire microservice ecosystem.

Production-Readiness Automation

Architecture reviews, audits, and roadmaps solve the challenges of microservice understanding at the developer and team levels, but understanding at an organizational level requires an additional component. As I've presented it so far, all of the work that goes into building a production-ready microservice is mostly manual, requiring developers to individually follow each audit step, make tasks and lists and roadmaps and check off individual requirement boxes. Manual work like this often gets put on the back burner to join the rest of the technical debt, even in the most productive and production-readiness driven teams.

One of the key principles of software engineering in practice is this: if you have to do something manually more than once, automate it so that you never have to do it again. This applies to operational work, it applies to any one-off, ad hoc situations and anything you need to type into a terminal, and not surprisingly, it applies to enforcing production-readiness standards across an engineering organization. Automation is the best onion you can give to your development teams.

It's easy to make a list of the production-readiness requirements for every microservice. I've done it myself at Uber, I've seen other developers implement the very same production-readiness standards in this book at their own companies, and I've created a template checklist ([Appendix A, *Production-Readiness Checklist*](#)) that you, the reader, can use. A list like this makes automating the checklist rather easy. For example, to check for fault tolerance and catastrophe-preparedness, you can run automated checks to ensure that the proper resiliency tests are in place, are running, and that each microservice passes the tests with flying colors.

The difficulty in automating each of these production-readiness checks will depend entirely on the complexity of your internal services within each layer of the microservice ecosystem. If all microservices and self-service tools have decent APIs, automation is a breeze. If your services have trouble communicating, or if any self-service internal tools are finicky or poorly written, you're going to have a bad time (and not just with produc-

tion-readiness, but with the integrity of your service and the entire microservice ecosystem).

Automating production-readiness increases organizational understanding in several extremely important and effective ways. If you automate these checks and run them constantly, teams in the organization will always know where each microservice stands. Publicize these results internally, give each microservice a production-readiness score measuring how production-ready their service is, require business-critical services to have a high minimum production-readiness score, and gate deployments. Production-readiness can be made part of the engineering culture, and this is one surefire way you can accomplish that.

Evaluate Your Microservice

Now that you have a better understanding of documentation, use the following list of questions to assess the production-readiness of your microservice(s) and microservice ecosystem. The questions are organized by topic, and correspond to the sections within this chapter.

Microservice Documentation

- Is the documentation for all microservices stored in a centralized, shared, and easily accessible place?
- Is the documentation easily searchable?
- Are significant changes to the microservice accompanied by updates to the microservice's documentation?
- Does the microservice's documentation contain a description of the microservice?
- Does the microservice's documentation contain an architecture diagram?
- Does the microservice's documentation contain contact and on-call information?
- Does the microservice's documentation contain links to important information?
- Does the microservice's documentation contain an onboarding and development guide?
- Does the microservice's documentation contain information about the microservice's request flow, endpoints, and dependencies?

- Does the microservice's documentation contain an on-call runbook?
- Does the microservice's documentation contain an FAQ section?

Microservice Understanding

- Can every developer on the team answer questions about the production-readiness of the microservice?
- Is there a set of principles and standards that all microservices are held to?
- Is there an RFC process in place for every new microservice?
- Are existing microservices reviewed and audited frequently?
- Are architecture reviews held for every microservice team?
- Is there a production-readiness audit process in place?
- Are production-readiness roadmaps used to bring the microservice to a production-ready state?
- Do the production-readiness standards drive the organization's OKRs?
- Is the production-readiness process automated?

[Support](#) [Sign Out](#)